what is ggplot?

➜ *code yourself*

what is dplyr?

➜ *code yourself^2*

ggplot architecture

visual defaults for basic fonts, colors, outlines

fit data into a plot “frame”

summarizes data into geoms instead of raw values

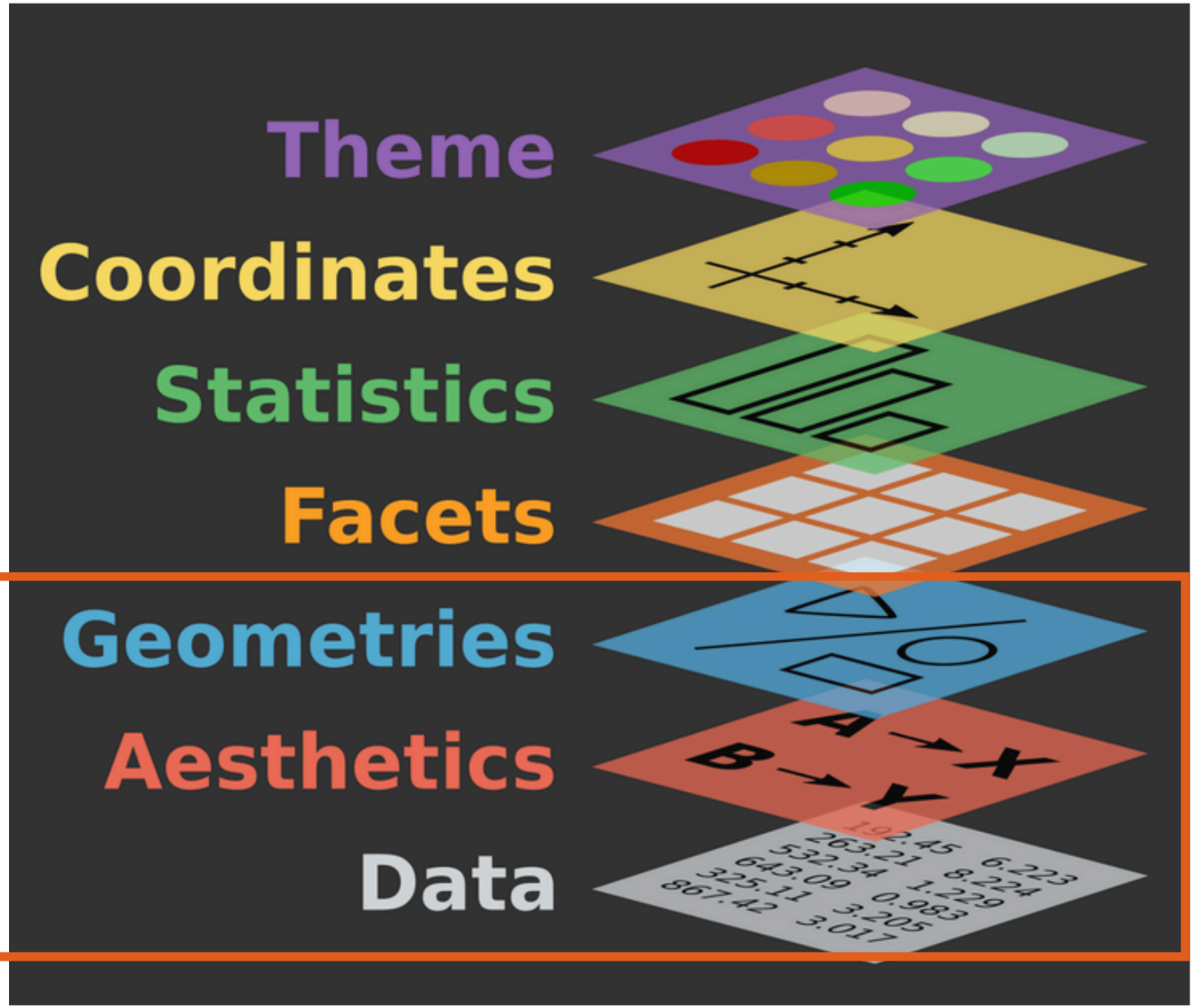
split plot based on variable

(geoms) list of objects that are going to plot following aesthetics

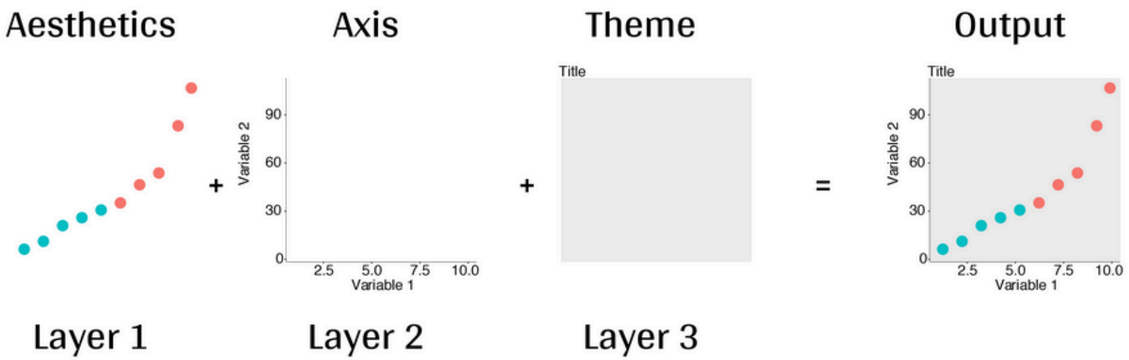
(aes) set of rules that link your data to the plot

underlying dataset (tidy format)

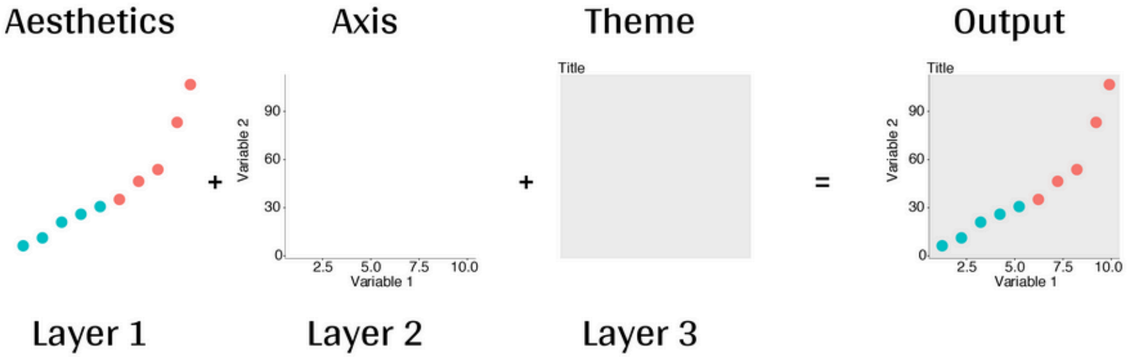
the most important



1. `ggplot2` mechanics



`ggplot2` mechanics



Here are the basic requirements to draw the simplest `ggplot`:

```
ggplot(data = <DATA>) +
    <GEOM function>(mapping=aes(<mappings>),
                    stat = <STAT>, position = <POSITION> ) +
    <COORDINATE function> +
    <SCALE function> +
    <THEME function> +
    <FACET function> +...
```

Required

Not required

You generally want to have your data in a long format, meaning one row per observation

Example:

is this a long format?

Year	CarbonFootprintCH	CarbonFootprintDEU	CarbonFootprintFRA
2000	*3*	*5*	*7*
2005	*4*	*6*	*8*
2010	*5*	*7*	*9*

You generally want to have your data in a long format, meaning one row per observation

Example: **what about this?**

Year	CarbonFootprint	Country
2000	*3*	*CH*
2005	*4*	*CH*
2010	*5*	*CH*
2000	*5*	*DEU*
2005	*6*	*DEU*
2010	*7*	*DEU*

To best handle data in this format, I highly recommend using dplyr (part of tidyverse)

-> Harder to get started than base R but gives you very good habits of data handling

Today, we'll use datasets that are already in this format

We'll talk more about dplyr later…

iris dataset

```
#### First explorations with iris ####
irisDataset <- iris # load the iris dataset
```

```
> head(irisDataset)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
> str(irisDataset)
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

Is there a relation between the length & the width of the iris Sepal?

How can we graphically address this question with ggplot?

- Discuss what plots we can make

- make the simplest plot (data + aesthetics)
- add complexity with a color aesthetics
- add a smooth line using geom_smooth()

Now it’s your turn:

Answer this with a plot

Are the length of the Petal & Sepal related to each other, in the iris dataset?

You can use the slides here, the tutorial linked here and in the beginning of the code https://r4ds.hadley.nz/data-visualize.html, or stack overflow

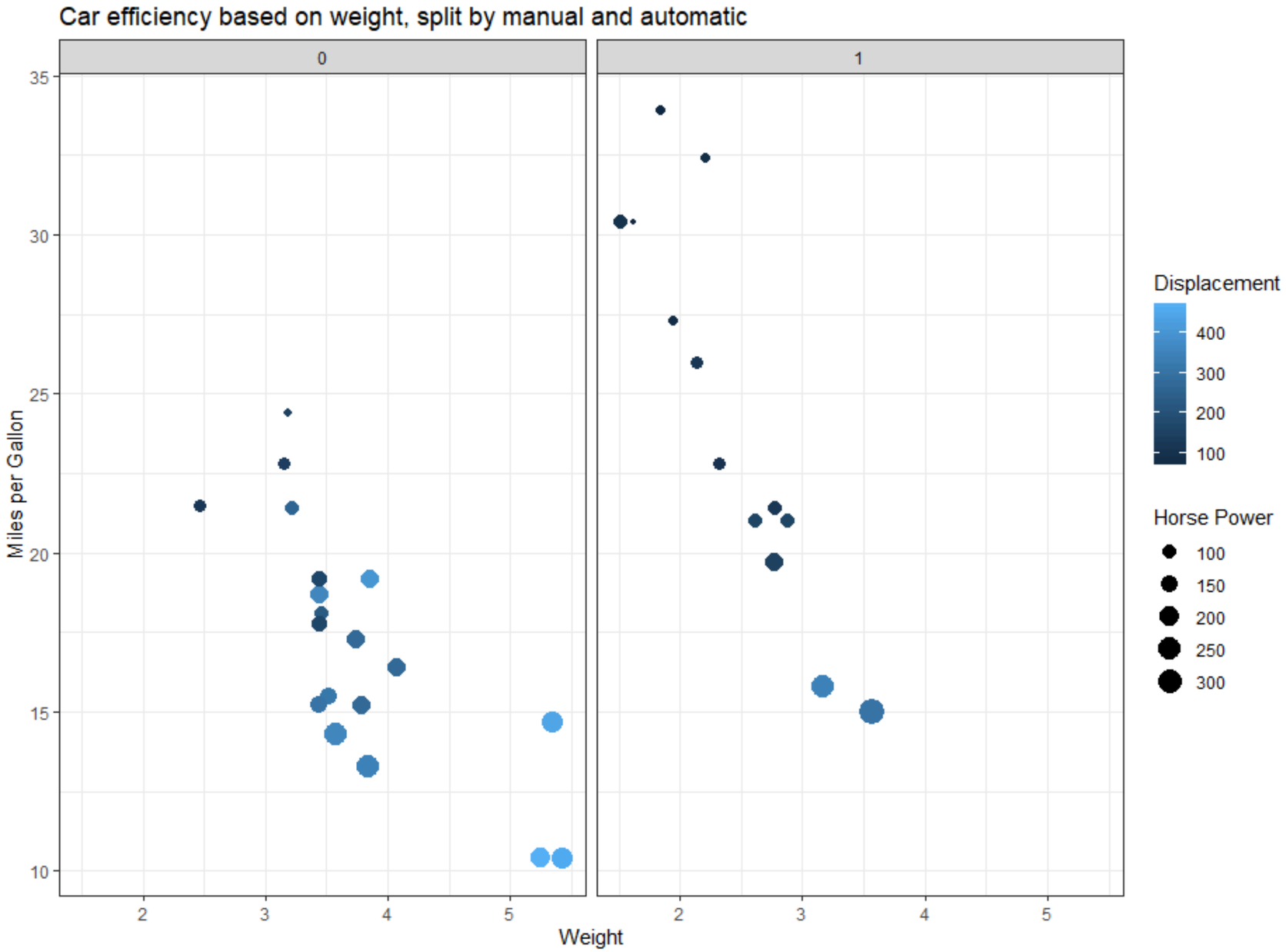
Try to debug errors yourself, using stack overflow, or asking Keith or I

#	Var	Description
0	name	Model of Vehicle
1	mpg	Miles/US Gallon
2	cyl	Number of cylinders
3	disp	Displacement (cu.in.)
4	hp	Gross horsepower
5	drat	Rear axle ratio
6	wt	Weight (lb/1000)
7	qsec	1/4 mile time
8	vs	V/S
9	am	Transmission Type
10	gear	Number of forward gears
11	carb	Number of carburetors

Displacement measures overall volume in the engine as a factor of cylinder circumference, depth and total number of cylinders. This metric gives a good proxy for the total amount of power the engine can generate.

A binary variable signaling whether vehicle has automatic (am=0) or manual (am=1) transmission configuration.

1. Describe the following plot

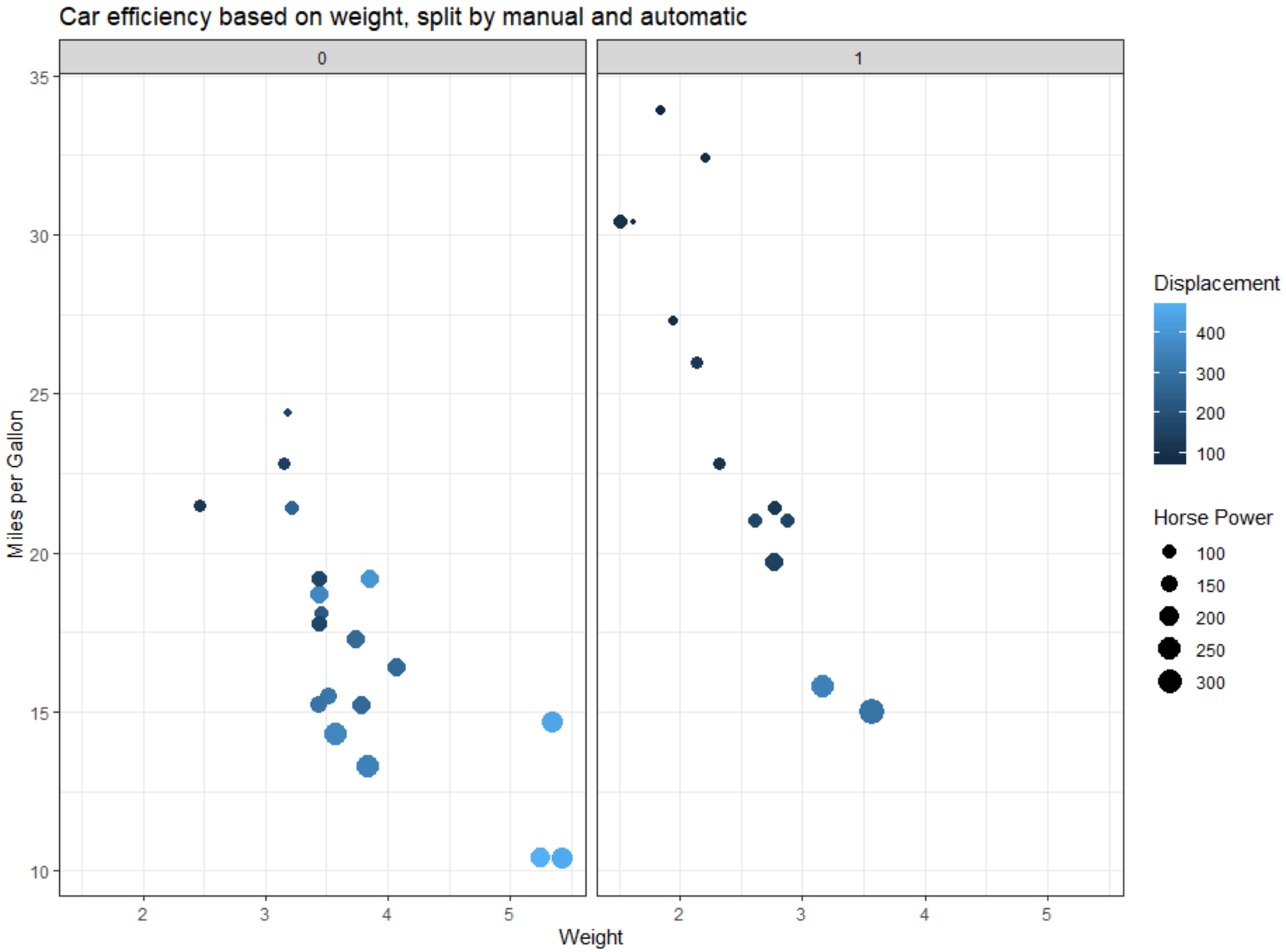


2. Replicate the plot in ggplot

theme_bw()

facet_wrap(~)

1. Describe the following plot

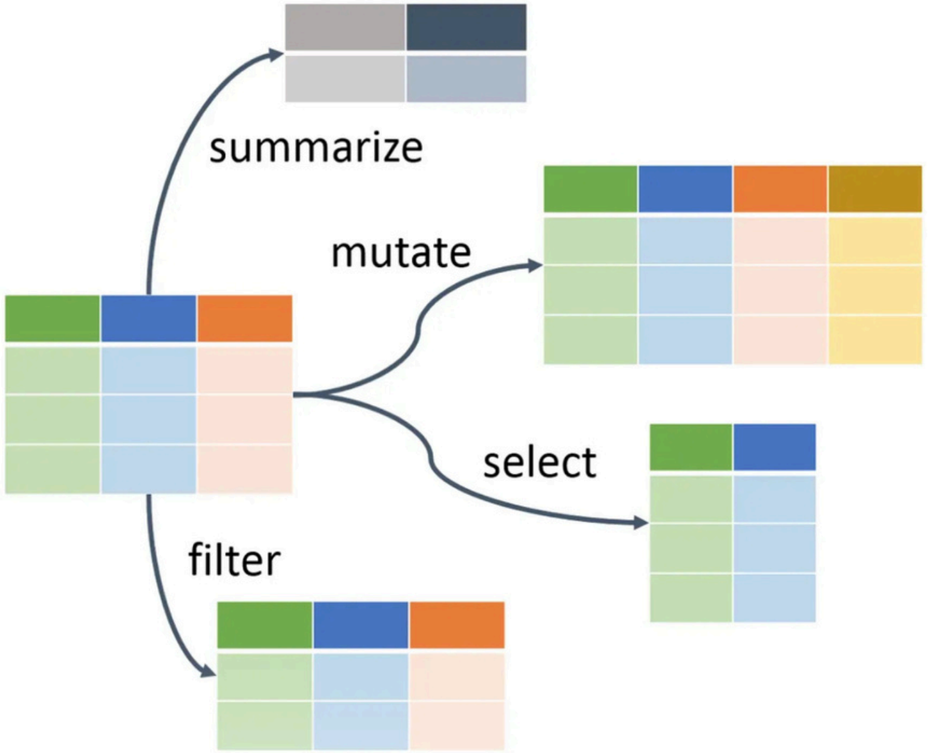


2. Replicate the plot in ggplot

theme_bw()

facet_wrap(~)


dplyr


summarize
mutate
select
filter

Cognitive process:

1. Take the **ydat** dataset, ***then***
2. **filter()** for genes in the leucine biosynthesis pathway, ***then***
3. **group_by()** the limiting nutrient, ***then***
4. **summarize()** to correlate rate and expression, ***then***
5. **mutate()** to round *r* to two digits, ***then***
6. **arrange()** by rounded correlation coefficients

The old way:

```
arrange(
  mutate(
    summarize(
      group_by(
        filter(ydat, bp=="leucine biosynthesis"),
      nutrient),
    r=cor(rate, expression)),
  r=round(r, 2)),
r)
```

The dplyr way:

```
ydat %>%
  filter(bp=="leucine biosynthesis") %>%
  group_by(nutrient) %>%
  summarize(r=cor(rate, expression)) %>%
  mutate(r=round(r,2)) %>%
  arrange(r)
```

https://hadley.github.io/genzplyr/index.html

aside from the memes, this has good examples for a basic dplyr workflow

Examples that slap

Basic workflow

Old way (cringe):

```
mtcars |>
  filter(mpg > 20) |>
  select(mpg, cyl, hp) |>
  mutate(kpg = mpg * 1.6) |>
  arrange(desc(mpg))
```

New way (bussin):

```
mtcars |>
  yeet(mpg > 20) |>
  vibe_check(mpg, cyl, hp) |>
  glow_up(kpg = mpg * 1.6) |>
  slay(desc(mpg))
```

Grouped summaries

Old way:

```
mtcars |>
  group_by(cyl) |>
  summarise(
    avg_mpg = mean(mpg),
    max_hp = max(hp)
  ) |>
  arrange(desc(avg_mpg))
```

New way (hits different):

```
mtcars |>
  squad_up(cyl) |>
  no_cap(
    avg_mpg = mean(mpg),
    max_hp = max(hp)
  ) |>
  slay(desc(avg_mpg))
```